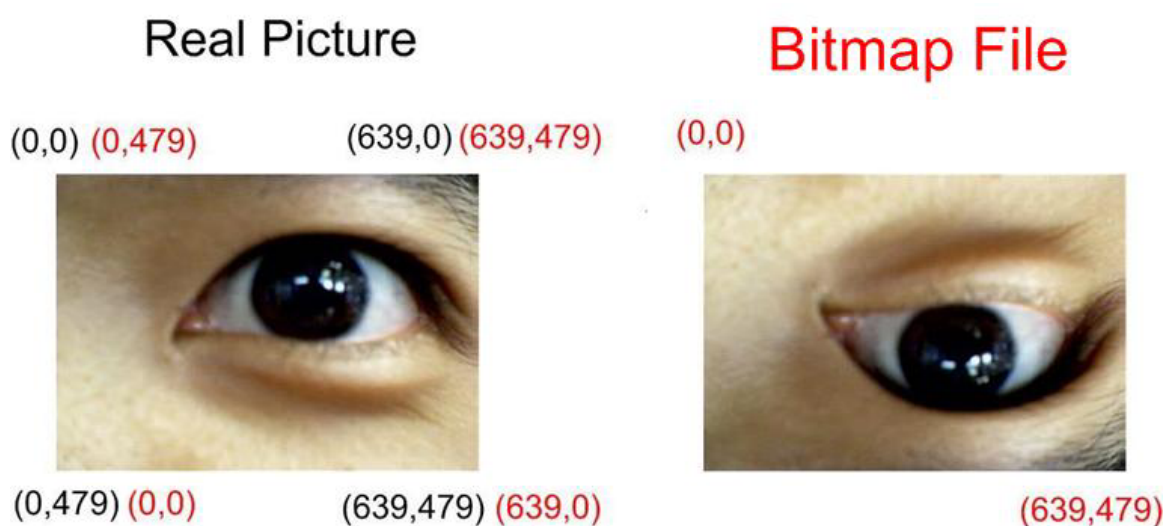


บทที่ 3

โครงสร้างและการจัดการรูปภาพ

3.1 โครงสร้างของภาพชนิด Bitmap

ภาพประเภท Bitmap Graphic นั้น เป็นรูปแบบหนึ่งของการเก็บภาพที่มีลักษณะเป็นพื้นฐานที่มีรูปแบบเรียบง่ายที่สุด โดยจะเก็บรูปเป็นลักษณะของสี่เหลี่ยม ซึ่งสามารถสร้างได้จากโปรแกรมวาดรูปทั่วไป โครงสร้างของภาพจะเก็บเป็นเมตริกซ์ของค่า pixel ของสีแต่ละจุดในหน่วยความจำ โดยสนับสนุนค่า pixel ที่เป็น 8 บิต และ 16 บิต (indirect) หรือ 24 บิต (direct) โดยสามารถเก็บอยู่ในไฟล์ชนิด .bmp โดยโครงสร้างของไฟล์ชนิด .bmp จะเก็บอยู่ในลักษณะที่ไม่เหมือนกับภาพที่แสดงผลออกมาโดยโปรแกรมวาดภาพต่างๆ ดังแสดงในรูปที่ 3-1 โดยที่ภาพต่างกันแกน Y ก็กลับตรงกันข้ามกัน



รูปที่ 3-1 แสดงลักษณะการเก็บภาพที่อยู่ในไฟล์เปรียบเทียบกับภาพที่แสดงจริงโดยใช้โปรแกรมต่างๆ

ในการเขียนโปรแกรมวิชวลเบสิก 6.0 เพื่อที่จะดึงภาพชนิด Bitmap ขึ้นมาใช้งานที่จะต้องมีส่วนขั้นตอนทำดังต่อไปนี้

1. เปิดไฟล์ที่ต้องการมาพิจารณาชนิดที่เป็น Bitmap

```
OpenFile = Text1.Text
```

```
Open OpenFile For Binary As #1
```

โดยทำการเปิดภาพขึ้นมาเป็นชนิดไบนารี (binary) โดยดึงชื่อไฟล์มาจากเท็กซ์บ็อกซ์ที่ชื่อว่า "Text1" มาใส่ตัวแปรชื่อ #1

2. ดึงส่วนหัว(header file)ของไฟล์ชนิด Bitmap มาใส่ในตัวแปรที่ h1,h2 ซึ่ง h2 จะการเก็บข้อมูลของขนาดของภาพเอาไว้

```
Dim h1 As BITMAPFILEHEADER
```

```
Dim h2 As BITMAPINFOHEADER
```

```
Dim XRes As Long 'Width pic
```

```
Dim YRes As Long 'Hight pic
```

```
Get #1, , h1
```

```
Get #1, , h2
```

```
XRes = h2.biWidth
```

```
YRes = h2.biHeight
```

นำขนาดของภาพมาใส่ในตัวแปรที่ชื่อว่า XRes ,YRes โดยที่ XRes คือ ขนาดความกว้างของภาพ และ YRes คือความสูงของภาพ

3. ดึงภาพมาใส่ตัวแปรชนิดอาร์เรย์ (Array) ซึ่งจะนำมาใช้ในการพิจารณาแต่ละจุดของภาพมีสีอะไร โดยแต่ละจุดจะประกอบด้วยค่า RGB ซึ่งก็คือค่าสี ประกอบด้วย แดง เขียว น้ำเงิน ตามลำดับ

```
Global RGBPalette() As Long 'Color Table
```

```
Global pic() As Long 'pixel
```

```
ReDim pic(XRes, YRes, 2) As Long
```

```
ReDim RGBPalette(255, 3) As Long
```

```
For i = 0 To YRes - 1
```

```
For j = 0 To XRes - 1
```

```
Get #1, , RGBentry
```

```
pic(j, i, 0) = Asc(Mid$(RGBentry, 3, 1))
```

```
pic(j, i, 1) = Asc(Mid$(RGBentry, 2, 1))
```

```
pic(j, i, 2) = Asc(Mid$(RGBentry, 1, 1))
```

```
Next
```

```
Next
```

โดยเราจะนำค่าสีในแต่ละจุดมาใส่ในตัวแปรที่ชื่อว่า "pic" ซึ่งเป็นอาร์เรย์ขนาดเท่ากับขนาดความใหญ่ของภาพ

4. เราสามารถดึงค่าสีของแต่ละจุดมาพิจารณาเพื่อเป็นพื้นฐานขั้นต้นของการหาจุดศูนย์กลางของดวงตาค่อนที่จะนำไปใช้กับภาพเคลื่อนไหวโดยจะกล่าวขั้นตอนการหาจุดศูนย์กลางของดวงตาและกำจัดสิ่งรบกวนเพื่อให้ได้จุดศูนย์กลางที่ถูกต้องในบทที่ 8,9 ต่อไป ตามลำดับ

หากท่านต้องการทราบการเก็บข้อมูลชนิด Bitmap อย่างละเอียด ท่านสามารถศึกษาได้จากส่วนที่เป็นภาคผนวก ก ที่ท้ายเล่ม

3.2 การสร้างพื้นที่ในหน่วยความจำเพื่อเก็บภาพ

ในการดึงภาพขึ้นมานั้นเราอาจไม่จำเป็นต้องสร้างตัวแปรชนิดอาร์เรย์(Array) มาเก็บภาพเพื่อที่จะนำมาพิจารณา แต่เราสามารถสร้างเนื้อที่ในหน่วยความจำมาเก็บภาพ ซึ่งจะมีสะดวกและความรวดเร็วต่อการทำงานของโปรแกรมมากขึ้น

ซึ่งเนื้อที่ในหน่วยความจำที่สร้างเพื่อเก็บรูปนั้นจะมีความสัมพันธ์กับการแสดงภาพบนจอ โดยการแสดงภาพบนจอที่โปรแกรมจะไม่สามารถเข้าถึงโดยตรงกับอุปกรณ์ได้(Hardware Device) เราต้องเขียนภาพผ่าน “Generic Drawing Function” ซึ่งเป็น API ตัวหนึ่ง คือ “Graphic Device Interface หรือ GDI” ซึ่ง GDI จะทำหน้าที่แปลงเป็น Hardware Command ให้

โดยการวาดรูปนั้นเราจะต้องวาดรูปลงบน Device Context (DC) ซึ่งจะเป็นตัวแทนของอุปกรณ์ โดยจะเปรียบเสมือนผืนผ้าที่เราใช้สำหรับวาดรูปไป



รูปที่ 3-2 การโหลดภาพมาเก็บในหน่วยความจำ

ขั้นตอนในการสร้างพื้นที่ในหน่วยความจำเพื่อเก็บภาพ

1. ประกาศคำสั่งที่ใช้ในการจัดการพื้นที่ในหน่วยจำที่ใช้สำหรับเก็บภาพ

ก่อนจะใช้ฟังก์ชันได้จะต้องมีการประกาศใช้ตอนต้นของโปรแกรมก่อน โดยจะอธิบายแต่ละฟังก์ชัน ดังนี้

Private Declare Function GetDC Lib "user32" (ByVal hwnd As Long) As Long

เป็นฟังก์ชันสำหรับการสร้างตัวชี้(Pointer) ให้ชี้ไปยัง DC นั้น เพื่อใช้ในการจัดการกับ DC ของส่วนที่นำออกมาเช่น ของจอภาพ ของพื้นที่ที่ใช้ในการแสดงรูป ในที่นี้จะใช้ในการดึงภาพจากพื้นที่ของ Component ของ ezVidCap ซึ่งเป็นภาพจริงจากกล้อง Web Cam

Private Declare Function DeleteDC Lib "gdi32" (ByVal hdc As Long) As Long

เป็นฟังก์ชันสำหรับลบค่าที่ใช้ในการชี้ Device Context ตัวนั้นอยู่

Private Declare Function CreateCompatibleDC Lib "gdi32" (ByVal hdc As Long) As Long

เป็นฟังก์ชันสำหรับการสร้างพื้นที่ในหน่วยความจำที่ในที่นี้เป็นObject ที่สอดคล้องกับDCของจอภาพ หรือพื้นที่ของ Component ของ ezVidCap

Private Declare Function CreateCompatibleBitmap Lib "gdi32" (ByVal hdc As Long, ByVal nWidth As Long, ByVal nHeight As Long) As Long
เป็นฟังก์ชันสำหรับการสร้าง Object ที่สอดคล้องกับภาพชนิด Bitmap

Private Declare Function SelectObject Lib "gdi32" (ByVal hdc As Long, ByVal hObject As Long) As Long
เป็นฟังก์ชันสำหรับเลือก Object เข้าไปใน Object ของ DC ในที่นี้ คือ ให้ของพื้นที่ในหน่วยความจำเพื่อใช้ในการเก็บรูปชนิด Bitmap

Private Declare Function DeleteObject Lib "gdi32" (ByVal hObject As Long) As Long
เป็นฟังก์ชันสำหรับลบ Object นั้นทิ้งไป

Private Declare Function SetPixel Lib "gdi32" (ByVal hdc As Long, ByVal x As Long, ByVal y As Long, ByVal crColor As Long) As Long
เป็นฟังก์ชันสำหรับใส่(Set) ค่าสีลงไปในหน่วยความจำเฉพาะจุดที่เราต้องการแก้ไขเปลี่ยนแปลงสีของจุดนั้น

Private Declare Function GetPixel Lib "gdi32" (ByVal hdc As Long, ByVal x As Long, ByVal y As Long) As Long
เป็นฟังก์ชันสำหรับดึง(Get) ค่าสีออกมาจากหน่วยความจำเฉพาะจุดที่เราต้องการทราบเพื่อพิจารณาค่าสีของจุดนั้น

2. ต้องมีการสร้างพื้นที่ในหน่วยความจำ(Memory DC) ซึ่งสอดคล้องกับจอภาพ (Screen DC) และใช้ในการเก็บภาพประเภท Bitmap

โดยเราจะต้องมีการสร้างพื้นที่ในหน่วยความจำเพื่อเก็บภาพ โดยจะต้องสอดคล้องกับพื้นที่ที่จะแสดงรูปละมีชนิดของรูปภาพเป็น Bitmap

Dim hDC, hBMP As Long

hDC = CreateCompatibleDC(PictureB.hdc)

hBMP = CreateCompatibleBitmap(PictureB.hdc, 256, 192)

SelectObject hDC, hBMP

ในที่นี้ต้องการแสดงภาพไปยัง Picture Box ที่ชื่อว่า "PictureB" โดยจะต้องสร้างพื้นที่ให้สอดคล้องด้วยโดยตัวแปรที่ใช้จัดการคือ hDC และ สร้างObject ชนิด Bitmap ซึ่งมีขนาด 256x192 แล้วเลือกให้พื้นที่ในหน่วยความจำ(hDC) เอาไว้เก็บภาพชนิดBitmapและมีขนาด 256x192

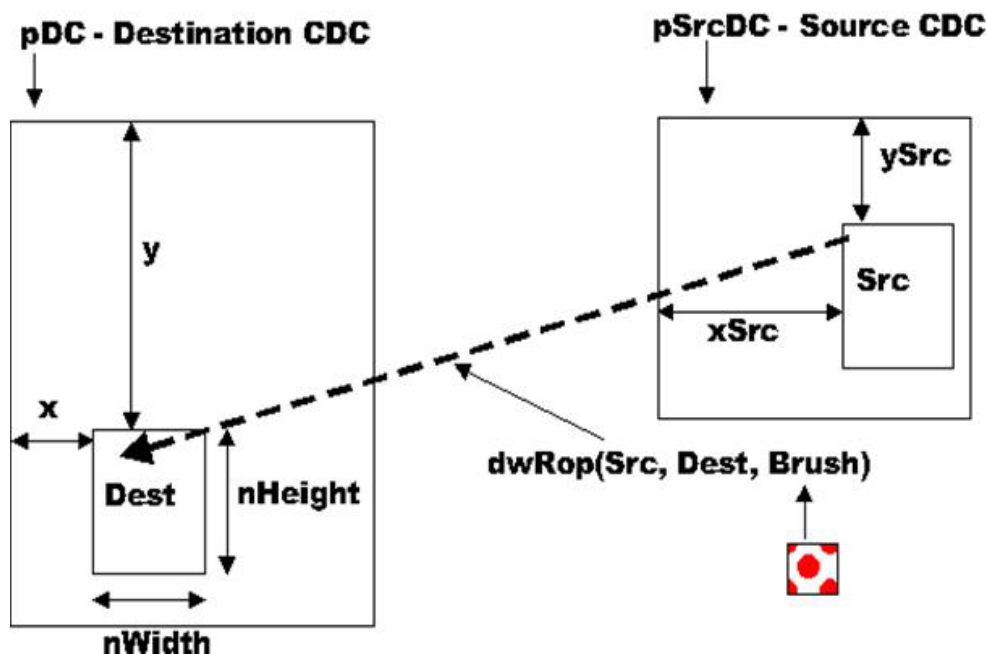
3.3 การโคลนหรือส่งรูปภาพมาเก็บไว้ในหน่วยความจำที่ได้สร้างไว้

ในการส่งรูปลงในหน่วยความจำที่จองไว้ นั้นเราจะใช้ฟังก์ชันที่ชื่อว่า BitBlt (Bitwise Block Transfer) ซึ่งจะทำให้การส่งค่าทั้งหมดไปยังเนื้อที่ในหน่วยความจำที่จองไว้ โดยที่ไม่ต้องไล่ส่งไปที่ละจุด (จะ Copy ระหว่าง Memory กับ Memory) ซึ่งจะมีความรวดเร็วอย่างมาก

โดยเราจะต้องประกาศในตอนต้นของโปรแกรมก่อนใช้งาน

Private Declare Function BitBlt Lib "gdi32" (ByVal hDestDC As Long, ByVal x As Long, ByVal y As Long, ByVal nWidth As Long, ByVal nHeight As Long, ByVal hSrcDC As Long, ByVal xSrc As Long, ByVal ySrc As Long, ByVal dwRop As Long) As Long

nRc = pDC->BitBlt(x, y, nWidth, nHeight, pSrcDC, xSrc, ySrc, dwRop);



รูปที่ 3-3 แสดงการส่งภาพของฟังก์ชัน BitBlt

ในการทำงานของฟังก์ชัน BitBlt จะต้องระบุตำแหน่งภาพของภาพต้นทางว่าจะเริ่มที่ไหน และ ตำแหน่งภาพของภาพปลายทางว่าเริ่มที่ไหน โดยที่ขนาดของภาพต้นทางและปลายทางจะมีขนาดเท่ากัน ดังจะแสดงในรูปที่ 3-3

โดยตัวอย่างของการใช้ฟังก์ชัน BitBlt คือ

```
hDC = GetDC(ezVidCap1.hwnd)
```

```
Dim MemDC, MemBMP As Long
```

```
MemDC = CreateCompatibleDC(PictureB.hdc)
```

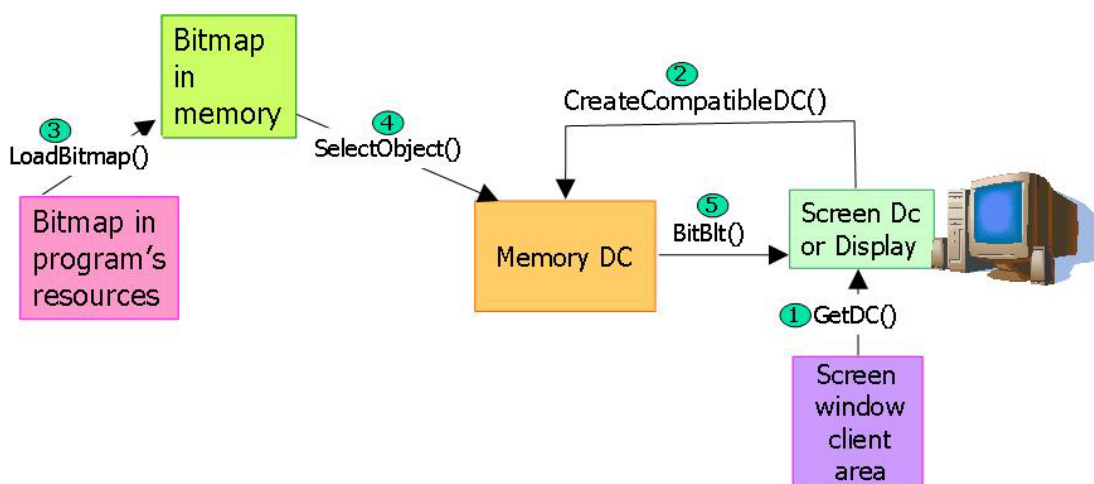
```
MemBMP = CreateCompatibleBitmap(PictureB.hdc, 256, 192)
```

```
SelectObject MemDC, MemBMP
```

```
BitBlt MemDC, 0, 0, Picture1.ScaleWidth, Picture1.ScaleHeight, hDC, 0, 0, vbSrcCopy
```

'No Zoom

โดยเราจะทำการสร้างตัวชี้ไปยังพื้นที่ของภาพจริงที่รับมาจากกล้องจากนั้นนำมาใส่ในหน่วยคำจำที่ชื่อว่า “MemDC” ดังที่แสดงอยู่ในรูปที่ 3-4 แสดงขั้นตอนการส่งรูปภาพมาใส่ในหน่วยความจำที่จองไว้

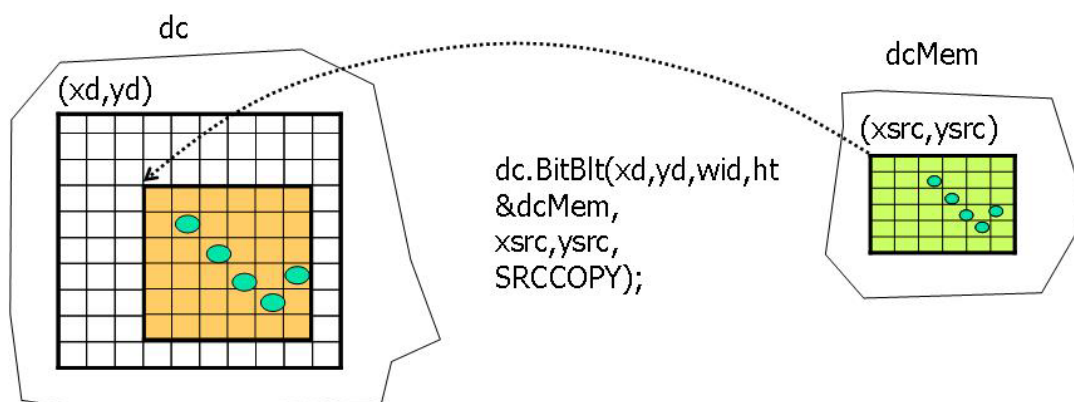


รูปที่ 3-4 แสดงขั้นตอนการส่งรูปภาพมาใส่ในหน่วยความจำที่จองไว้

เราอาจจะใช้ฟังก์ชัน BitBlt ในการโหลดค่าหรือส่งค่าจากหน่วยความจำมาแสดงผลบนพื้นที่ที่สร้างไว้บนหน้าจอของแอปพลิเคชันก็ได้ แสดงในรูปที่ 3-5

A CClientDC device Context, dc
The visible screen

A memory Device Context



รูปที่ 3-5 แสดงการโหลดหรือส่งรูปภาพมาแสดงผลบนพื้นที่ที่สร้างไว้บนหน้าจอของแอปพลิเคชัน

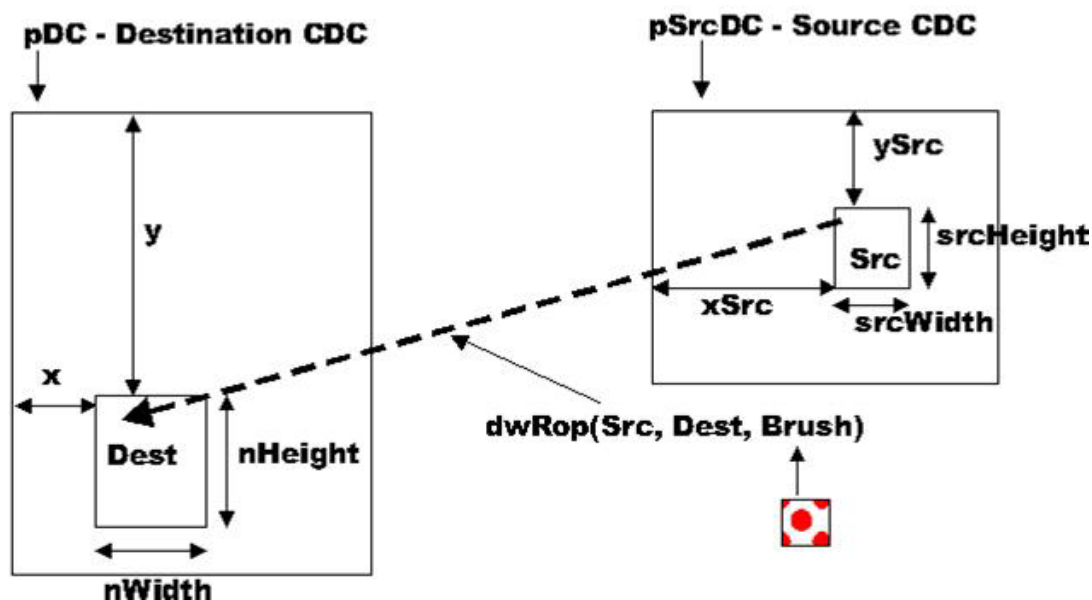
3.4 การลดหรือขยายขนาดของรูปภาพในหน่วยความจำ

เราสามารถลดหรือขยายขนาดของภาพได้โดยใช้ฟังก์ชัน StretchBlt โดยที่ไม่ต้องใช้ทรัพยากรขยายภาพแบบ First Order โดยการทำงานของฟังก์ชัน StretchBlt นั้นจะเหมือนกับ BitBlt เพียงแต่จะสามารถขยายหรือลดขนาดของภาพได้

โดยเราจะต้องประกาศในตอนต้นของโปรแกรมก่อนใช้งาน

Declare Function StretchBlt Lib "gdi32" (ByVal hdc As Long, ByVal x As Long, ByVal y As Long, ByVal nWidth As Long, ByVal nHeight As Long, ByVal hSrcDC As Long, ByVal xSrc As Long, ByVal ySrc As Long, ByVal nSrcWidth As Long, ByVal nSrcHeight As Long, ByVal dwRop As Long) As Long

nRc = pDC->StretchBlt(x, y, nWidth, nHeight, pSrcDC, xSrc, ySrc, srcWidth, srcHeight, dwRop);



รูปที่ 3-6 แสดงการส่งภาพของฟังก์ชัน StretchBlt

ในการทำงานของฟังก์ชัน StretchBlt จะต้องระบุตำแหน่งภาพของภาพต้นทางว่าจะเริ่มที่ไหน และตำแหน่งภาพของภาพปลายทางว่าเริ่มที่ไหนเช่นกัน เพียงแต่โดยที่ขนาดของภาพต้นทางแลปลายทางนั้นจะมีขนาดไม่เท่ากัน โดยเราสามารถระบุได้ว่าภาพต้นทางมีขนาดเท่านี้ ภาพปลายทางจะให้มีความเท่าไรได้ โดยจะแสดงในรูปที่ 3-6